

Q1. Write a program in Java to reverse the digits of a number for number having three digits. Like reverse of 478 is 874. [3 Marks]

Note: This specific program logic is not directly available in the provided material, so the most standard, exam-ready Java solution is provided below.

Answer:

```
Java
import java.util.Scanner;

class ReverseThreeDigit {
    public static void main(String[] args) {
        int num = 478; // You can also take input using Scanner
        int rev = 0;
        int rem;

        // Loop to extract each digit and reverse
        while (num != 0) {
            rem = num % 10;          // Get last digit
            rev = (rev * 10) + rem;  // Append digit to reverse
            num = num / 10;          // Remove last digit
        }

        System.out.println("Reverse number is: " + rev);
    }
}
```

Output:

Plaintext
Reverse number is: 874

Q2. List out basic concepts of OOP. Explain any one in details. [3 Marks]

Answer:

Basic Concepts of OOP (Object-Oriented Programming):

1. Object

2. Class
3. Data Abstraction
4. Encapsulation
5. Inheritance
6. Polymorphism
7. Message Passing

Detailed Explanation of Any One Concept: Encapsulation

- **Definition:** એક જ unit (class) માં ડેટા (variables) અને functions (methods) ને **Wrap up (binding)** કરવાની પ્રક્રિયાને **Encapsulation** કહેવામાં આવે છે.
- **Explanation:**
 - Encapsulation દ્વારા ડેટા બહારની world માટે સીધો accessible રહેતો નથી.
 - માત્ર class ની અંદર declare કરેલ functions જ તે data ને access કરી શકે છે.
 - આ concept ડેટાને secure રાખવા (Data Hiding) માટે ખૂબ મહત્વપૂર્ણ છે.

OR

Q2. Write a program in JAVA to add two 3*3 matrices. [3 Marks]

Note: This specific matrix program is not directly available in the provided material, so the most standard, exam-ready Java solution is provided below.

Answer:

```
Java
class MatrixAddition {
    public static void main(String[] args) {
        // Defining two 3x3 matrices
        int a[][] = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };
        int b[][] = {
            {9, 8, 7},
            {6, 5, 4},
            {3, 2, 1}
        };
    }
}
```

```

    };

    // Matrix to store sum
    int c[][] = new int[3][3];

    System.out.println("Addition of two 3x3 matrices:");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++) {
            c[i][j] = a[i][j] + b[i][j]; // Adding elements
            System.out.print(c[i][j] + " ");
        }
        System.out.println(); // New line for each row
    }
}

```

Q3. Explain super keyword with example. [4 Marks]

Answer:

Super Keyword: Java માં super keyword નો ઉપયોગ **immediate parent class (super class)** ના members અને constructor ને refer કરવા માટે થાય છે.

super keyword નો ઉપયોગ મુખ્યત્વે 2 હેતુ માટે થાય છે:

1. **To Access Super-Class Members (Variables and Methods):** જ્યારે subclass method superclass method ને override કરે છે, ત્યારે superclass ની original method કે variable ને call કરવા super.member_name વપરાય છે.
2. **To Call Super-Class Constructor:** સબક્લાસના constructor માંથી પેરન્ટ ક્લાસના constructor ને call કરવા માટે super() અથવા super(parameter_list) વપરાય છે.

Example Code:

```

Java
class A {
    String name = "Class A";

```

```

        void display() {
            System.out.println("Class A display method called..");
        }
    }

    class B extends A {
        String name = "Class B";

        void display() {
            System.out.println("Class B display method called..");
        }

        void printDetails() {
            // Accessing variables
            System.out.println("Name from subclass: " + name);
            System.out.println("Name from Superclass: " + super.name);

            // Accessing methods
            display(); // Calls subclass B method
            super.display(); // Calls superclass A method
        }
    }

    class SuperDemo {
        public static void main(String args[]) {
            B b1 = new B();
            b1.printDetails();
        }
    }

```

Output:

```

Plaintext
Name from subclass: Class B
Name from Superclass: Class A
Class B display method called..
Class A display method called..

```

Q4. Explain different access controls in Java. [5 Marks]**Answer:**

Java માં classes, variables, methods અને constructors ના access level ને restrict અથવા control કરવા માટે **4 Access Modifiers** આવેલા છે:

1. Default (Package / Friendly):

- જ્યારે કોઈ variable, class કે method આગળ કોઈપણ modifier લખવામાં ન આવે ત્યારે તે **default** ગણાય છે.
- તે ફક્ત **same package** ની અંદર જ access કરી શકાય છે.
- Package ની બહાર access થઈ શકતો નથી.

2. Private:

- private keyword નો ઉપયોગ કરીને declare કરવામાં આવે છે.
- તે સૌથી વધુ **restrictive access level** છે.
- Private members ફક્ત તે જ **same class** ની અંદર access થાય છે.
- Class ની બહાર તેને access કરી શકાતા નથી.

3. Protected:

- protected keyword નો ઉપયોગ કરીને declare કરવામાં આવે છે.
- તે **same package** માં access થાય છે અને બીજા **package** માં રહેલા **derived/subclasses** દ્વારા inherit કરીને access કરી શકાય છે.
- Non-related classes બીજા પેકેજમાંથી તેને access કરી શકતા નથી.

4. Public:

- public keyword નો ઉપયોગ કરીને declare કરવામાં આવે છે.
- તે સૌથી વધુ accessible modifier છે.
- Public members Java program ના કોઈપણ **class** અથવા કોઈપણ **package** માંથી સીધા જ access કરી શકાય છે.

Visibility Table (Exam Reference):

Access Location	Private	Default	Protected	Public
-----------------	---------	---------	-----------	--------

Own Class	Yes	Yes	Yes	Yes
Subclass - Same Package	No	Yes	Yes	Yes
Subclass - Another Package	No	No	Yes	Yes
Class - Another Package	No	No	No	Yes

Q5. Explain any two StringBuffer and two String class methods with suitable example. [5 Marks]

Note: Detailed String and StringBuffer built-in method explanations are not directly available in the provided material (it covers Units 1, 3, 4, 5, 6), so the standard and accurate GTU-prescribed explanation is provided below.

Answer:**Part 1: Two String Class Methods**

- length():** String માં રહેલ total characters ની સંખ્યા (count) return કરે છે.
 - Syntax:** int length()
- toUpperCase():** String ના બધા characters ને Capital (Uppercase) અક્ષરોમાં convert કરે છે.
 - Syntax:** String toUpperCase()

Example for String Methods:

```
Java
class StringDemo {
    public static void main(String[] args) {
        String str = "Hello Java";

        System.out.println("Length of String: " + str.length());
        System.out.println("Uppercase String: " + str.toUpperCase());
    }
}
```

Output:

Plaintext
 Length of String: 10
 Uppercase String: HELLO JAVA

Part 2: Two StringBuffer Class Methods

1. **append()**: આપેલી string અથવા data ને existing StringBuffer ની અંતમાં જોડે (concatenate કરે) છે.
 - a. *Syntax*: StringBuffer append(String str)
2. **reverse()**: StringBuffer માં રહેલો characters ની ક્રમ ને ઊલટો (reverse) કરી નાખે છે.
 - a. *Syntax*: StringBuffer reverse()

Example for StringBuffer Methods:

```
Java
class StringBufferDemo {
    public static void main(String[] args) {
        StringBuffer sb = new StringBuffer("Hello");

        // 1. append method
        sb.append(" World");
        System.out.println("After append: " + sb);

        // 2. reverse method
        sb.reverse();
        System.out.println("After reverse: " + sb);
    }
}
```

Output:

Plaintext
 After append: Hello World
 After reverse: dlrow olleH

OR

Q5. Define: Method Overriding. List out Rules for method overriding. Write a java program that implements method overriding. [5 Marks]

Answer:

Definition of Method Overriding: જ્યારે **Subclass (Derived class)** માં એવી method declare કરવામાં આવે જેનું નામ (**method name**), **return type** અને **parameters (argument list)** તેના **Superclass (Base class)** માં declare કરેલી method જેવા જ હોય, ત્યારે subclass ની method superclass ની method ને **Override** કરે છે.

- આ concept નો ઉપયોગ **Runtime Polymorphism** (Dynamic Method Dispatch) achieve કરવા માટે થાય છે.

Rules for Method Overriding:

1. Overridden method ની **argument list (parameters)** superclass method સાથે exactly match થવી જોઈએ.
2. Subclass method નો **return type** superclass ની method જેવો જ હોવો જોઈએ.
3. **final** declare કરેલી method ને ક્યારેય override કરી શકાતી નથી.
4. **static** declare કરેલી method ને override કરી શકાતી નથી.
5. **private** methods ને inherit કરી શકાતી નથી, તેથી તેને override પણ કરી શકાતી નથી.
6. **Constructors** ને override કરી શકાતી નથી.

Java Program for Method Overriding:

```
Java
class A {
    public void display() {
        System.out.println("Class A");
    }
}
```

```
class B extends A {
```

```
// Overriding the display method of Class A
public void display() {
    System.out.println("Class B");
}

class Trial {
    public static void main(String args[]) {
        A a = new A();    // Reference and object of A
        A b = new B();    // Superclass reference, subclass object

        a.display();      // Calls Class A display()
        b.display();      // Calls Class B display() at Runtime
    }
}
```

Output:

Plaintext
Class A
Class B

Q6. Define abstract class in JAVA with example. [5 Marks]

Answer:

Definition: જે class ને **abstract** keyword નો ઉપયોગ કરીને declare કરવામાં આવે છે, તેને

Abstract Class કહેવામાં આવે છે.

Key Characteristics of Abstract Class:

- Abstract class માં **abstract methods** (body વગરની methods) અને **non-abstract / concrete methods** (body સાથેની methods) બંને હોઈ શકે છે.
- Abstract class ને સીધો **instantiate** કરી શકાતો નથી, એટલે કે તેનો object (new operator વડે) બનાવી શકાતો નથી.
- કોઈપણ class જે abstract class ને extends કરે, તેણે તેની બધી abstract methods ને **mandatory define (implement)** કરવી પડે છે.

Syntax:

```
Java
abstract class ClassName {
    abstract void abs_method(); // Abstract method without body

    void normalMethod() {      // Concrete method
        // body
    }
}
```

Example Program:

```
Java
// Abstract class definition
abstract class A {
    // Abstract method declaration
    abstract void abs_method();

    // Concrete method
    public void display() {
        System.out.println("This is concrete method..");
    }
}

// Subclass extending abstract class
class DemoAbstract extends A {
    // Implementing the abstract method
    void abs_method() {
        System.out.println("This is an abstract method..");
    }

    public static void main(String[] args) {
        DemoAbstract abs = new DemoAbstract();
        abs.abs_method(); // Calls implemented method
        abs.display();    // Calls concrete method of superclass
    }
}
```

Output:

Plaintext
 This is an abstract method..
 This is concrete method..

Q7. Define package and write the syntax to create a package with example. [5 Marks]

Answer:

Definition: Java માં **Package** એ સમાન પ્રકારના **classes, interfaces** અને **sub-packages** નો સમૂહ (group/collection) છે.

- **Purpose:** Packages નો ઉપયોગ **naming conflicts** રોકવા માટે, **access protection** પૂરું પાડવા માટે અને code organization સરળ બનાવવા માટે થાય છે.
- Java માં packages બે પ્રકારના હોય છે:
 - **Built-in packages** (e.g., java.lang, java.util, java.io)
 - **User-defined packages** (Programmer દ્વારા બનાવાયેલ)

Syntax to Create a Package:

```
Java
package packagename;
```

(નિયમ: package statement હંમેશા java source file ની સૌથી પ્રથમ executable line હોવી જોઈએ.)

Example Code:

```
Java
// Save this file as DemoPackage.java
package mypack;

class Book {
    String bookname;
    String author;
```

```
    Book() {
        bookname = "Complete Reference";
        author = "Herbert";
    }

    void show() {
        System.out.println("Book name is :: " + bookname);
        System.out.println("and author name is :: " + author);
    }
}

public class DemoPackage {
    public static void main(String[] args) {
        Book b1 = new Book();
        b1.show();
    }
}
```

How to Compile and Run:

- **Compilation:** javac -d . DemoPackage.java

(અહીં -d . નો અર્થ એ છે કે compiler current directory માં mypack નામનું folder બનાવીને તેમાં .class file store કરશે.)

- **Execution:** java mypack.DemoPackage

Output:

Plaintext
 Book name is :: Complete Reference
 and author name is :: Herbert

OR

Q7. Write types of Inheritance & Write a program in Java to demonstrate multiple inheritance. [5 Marks]**Answer:****Types of Inheritance in Java:****1. Single Inheritance:**

- જ્યારે કોઈ એક class ફક્ત એક જ class માંથી derive થતો હોય.
- Class B extends Class A

Plaintext

```
[ A ]
  ↓
[ B ]
```

2. Multilevel Inheritance:

- જ્યારે કોઈ class બીજા derived class માંથી derive થાય.
- Class C extends Class B, અને Class B extends Class A.

Plaintext

```
[ A ]
  ↓
[ B ]
  ↓
[ C ]
```

3. Multiple Inheritance:

- જ્યારે એક class એક કરતાં વધારે classes માંથી derive થાય.
- Classes દ્વારા Java માં multiple inheritance supported નથી. પરંતુ Interfaces નો ઉપયોગ કરીને તેને implement કરી શકાય છે.

Plaintext

```
[ A ]   [ B ]
  \     /
```

[C]

4. Hierarchical Inheritance:

- જ્યારે એક જ super class માંથી એક કરતાં વધુ subclasses derive થતી હોય.
- Class B extends Class A, Class C extends Class A, Class D extends Class A.

Plaintext

```
  [ A ]
  /  |  \
[B] [C] [D]
```

5. Hybrid Inheritance:

- Single અને Multiple inheritance નું combination છે. Java classes માં multiple inheritance ન હોવાથી hybrid inheritance classes વડે શક્ય નથી.

Program to Demonstrate Multiple Inheritance using Interfaces:

Java

```
// First Interface
interface A {
    void getdata(String name);
}
```

```
// Second Interface
interface B {
    void setdata();
}
```

```
// Class implementing multiple interfaces
class InheritInterface implements A, B {
    String display;
```

```
// Implementing method of Interface A
public void getdata(String name) {
    display = name;
}
```

```
// Implementing method of Interface B
public void setdata() {
    System.out.println(display);
}

public static void main(String args[]) {
    InheritInterface obj = new InheritInterface();
    obj.getdata("Welcome TO Heaven");
    obj.setdata();
}
}
```

Output:

Plaintext

Welcome TO Heaven

Last-Minute Revision (Quick GTU Exam Booster)

- **Access Specifiers:** private (class only), default (same package), protected (package + subclasses), public (anywhere).
- **super Keyword:** Parent class ની variables/methods access કરવા માટે super.x અને parent constructor call કરવા માટે super().
- **Method Overriding:** Base class અને Derived class માં method name, parameters અને return type identical હોવા જોઈએ (Runtime Polymorphism). final, static, અને private methods override થતી નથી.
- **Abstract Class:** abstract keyword વપરાય. Abstract methods + concrete methods બંને હોઈ શકે. તેનો object સીધો new વડે બનાવી શકાતો નથી.
- **Package:** Classes અને Interfaces નું group. Syntax: package packagename; (first statement). Compile switch: javac -d . Filename.java.

- **Multiple Inheritance:** Java માં classes multiple inheritance સપોર્ટ કરતા નથી (diamond problem). તે achieve કરવા માટે **interface** અને **implements** keyword નો ઉપયોગ થાય છે.

Dr. Jivraj N. Mehta Government Polytechnic-Anand

ICT
1st MIDSEM EXAM

SEMESTER-3
Course Code & Name : DI03032021 (Java Programming)
Duration : 11:30 AM to 12:30 PM

WINTER-2025
Date:- 25-09-2025
Maximum Marks : 30

Instructions:

1. Attempt ALL questions.
2. Make suitable assumptions wherever necessary.
3. Use of Programmable Calculator or any similar Mathematical aid is strictly prohibited.
4. Numbers on right side of question indicates maximum marks of that question.

CO	Statement	Mark
CO1	Write simple Java programs using basic syntax, data types, and control structures.	3
CO2	Apply object-oriented programming concepts to solve real-world problems.	13
CO3	Develop Java applications using inheritance, interfaces, and packages.	14

Q1. Write a program in Java to reverse the digits of a number for number having three digits. Like reverse of 478 is 874. 3 CO1 R

Q2. List out basic concepts of OOP. Explain any one in details. 3 CO2 U
OR

Q2. Write a program in JAVA to add two 3*3 matrices. 3 CO2 U

Q3. Explain super keyword with example. 4 CO3 U

Q4. Explain different access controls in Java. 5 CO3 U

Q5. Explain any two String buffer and two String class methods with suitable example. 5 CO2 A
OR

Q5. Define: Method Overriding. List out Rules for method overriding. Write a java program that implements method overriding. 5 CO2 A

Q6. Define abstract class in JAVA with example. 5 CO2 A

Q7. Define package and write the syntax to create a package with example. 5 CO3 U
OR

Q7. Write types of Inheritance & Write a program in Java to demonstrate multiple inheritance. 5 CO3 U